

KuboEngine

自作エンジン上で制作した 3D サバイバルアクション

C++20 と DirectX 12 でゲームエンジンを構築し、その上で Title・Play・Result までを持つゲームを制作しました。描画基盤だけでなく、ゲーム進行、武器、敵、衝突判定、データ調整、デバッグ・検証まで一つのプロジェクトとして実装しています。

C++20	DirectX 12	HLSL	ImGui	CSV	Assimp
-------	------------	------	-------	-----	--------

作品概要

敵を倒して経験値を集め、レベルアップ時に武器や能力を選びながらボス戦まで生き残るゲームです。プレイ中に得たコインは Result で集計し、Title で恒久強化やキャラクター解放に使用します。

項目	内容
ゲーム構成	Title / Play / Result、通常戦闘、レベルアップ、ボス戦、恒久強化
エンジン	描画、入力、音声、モデル、スプライト、パーティクル、カメラ、シーン管理
ゲーム実装	Player、Enemy、Weapon、HUD、GameSession、デバッグ表示
検証	CPU 回帰テスト、シーン遷移ストレス、SRV テレメトリ

技術的な軸

設計	engine/ と game/directxgame/ を分離し、PlayScene から進行・武器・敵・HUD を責務ごとに切り出しました。
描画	FrameContext、Fence、SRV 再利用、ResourceBarrier まで DirectX 12 の寿命と状態を明示的に管理します。
負荷対策	OBB の前に空間分割で候補を絞り、上限値とテレメトリを組み合わせで破綻箇所を確認します。

01 ARCHITECTURE

エンジン層とゲーム層の分離

共通機能は engine/、作品固有の処理は game/directxgame/ に配置しています。起動処理では GameSceneFactory を Game へ渡し、main.cpp は起動と例外処理に絞りました。ゲーム固有のシーンを共通基盤から直接生成しない境界を置いています。

main.cpp	Game	GameSceneFactory	Title / Play / Result
----------	------	------------------	-----------------------

PlayScene を構成の起点にする

PlayScene は各機能を保持して接続しますが、処理そのものは責務ごとのクラスへ分けています。変更理由が異なる処理を同じクラスへ集めないことで、武器追加や UI 修正がシーン全体へ波及しにくい構成にしました。

クラス	役割
Framework	ゲームループと共通サービス
GameSceneFactory	Title、Play、Result の生成
PlayScene	ゲームプレイを構成する機能の保持と接続
GameplayFlowController	Start、Playing、Boss、Pause、LevelUp、Dead の状態管理
PlayerManager	HP、経験値、能力値、武器強化の窓口
EnemyManager	敵生成、EXP、衝突、ボス進行
GameSession	結果、コイン、恒久強化、キャラクター情報

ゲーム進行とシーン間データ

GameplayFlowController は通常更新、ボス演出、ポーズ、レベルアップ、死亡を状態として分離します。シーンをまたぐ値は GameSession に集約し、RunResult へ経験値、コイン、最終レベル、撃破数、経過フレームを記録します。各シーンが互いの内部状態を直接参照しないよう、共有データの置き場所を限定しました。

Play 結果を記録	Result 集計・表示	Title 強化・解放	Play 次のラン
---------------	-----------------	----------------	--------------

状態に応じたレベルアップ候補

LevelUpChoiceService は PlayerManager の状態から武器候補と能力候補を生成します。最大レベルの武器、上限到達済みの能力、HP 満タン時の回復を除外し、武器と能力が混ざらう候補数を調整してから表示順をシャッフルします。

5 武器	通常弾 / 周囲弾 / 雷撃 / 爆発弾 / ソード。強化値は weaponUpgradeSettings.csv から読み込みます。
------	---

02 IMPLEMENTATION

DirectX 12 のフレーム・リソース管理

CPU が次のフレームを準備している間も GPU が前のフレームを参照できるよう、2 つの FrameContext を用意しました。各コンテキストは CommandAllocator、Upload Arena、遅延解放リソース、Fence 値を保持します。GPU が参照する一時リソースを Fence 完了まで保持し、CPU 側の早すぎる再利用を防ぎます。

Frame 0 Upload / Fence	GPU 実行	Frame 1 Upload / Fence	完了後に再利用
---------------------------	--------	---------------------------	---------

ディスクリプタとリソース状態

SrvManager は Texture2D、CubeMap、StructuredBuffer、ShadowMap などの用途と使用数を記録し、解放されたディスクリプタを Fence 完了後に再利用します。DirectXCommon はリソース状態を追跡し、遷移元と遷移先から ResourceBarrier を発行します。追跡値と指定した遷移元が異なる場合は例外として検出します。

管理対象	実装
フレーム寿命	FrameContext / Upload Arena / deferred release / Fence
SRV 寿命	retired descriptor を Fence 完了後に再利用
状態遷移	全体・個別サブリソースの追跡と ResourceBarrier
追加バリア	UAV barrier / aliasing barrier

Shadow Map とオフスクリーン描画

Shadow Pass ではライト視点の深度マップを作成し、通常描画の Pixel Shader で SampleCmpLevelZero による 3×3 比較サンプリングを行います。開始時に深度テクスチャを DEPTH_WRITE、終了時に PIXEL_SHADER_RESOURCE へ遷移します。caster はライト範囲でカリングし、候補数・描画数・除外数を記録します。

PIXEL_SHADER_RESOURCE	DEPTH_WRITE Shadow Pass	PIXEL_SHADER_RESOURCE	通常描画で参照
-----------------------	----------------------------	-----------------------	---------

OBB と空間分割

通常弾、周囲弾、爆発弾、プレイヤー、敵の詳細判定には OBB を使用します。敵は衝突半径が重なるすべての XZ セルへ登録し、検索後に重複を除去してから OBB 判定へ進みます。セル境界で候補が漏れることを防ぎつつ、全敵との総当たりを避けます。

敵をセル登録	周辺セル検索	重複を除去	OBB 詳細判定	ダメージ / 押し戻し
計測条件	空間分割と Shadow Pass の性能値は未計測です。資料では実装と検証手段を説明し、FPS や GPU 時間の改善値は断定しません。			

03 VERIFICATION & NEXT

調整と検証を実装の一部にする

プレイヤー能力、武器強化、敵設定、スポーン、UI 配置、デバッグ調整値を CSV へ分離しました。数値変換では文字列全体を消費したか、有限値か、範囲内かを確認します。ImGui では敵数、弾数、パーティクル数、FPS、SRV 使用数、Shadow Pass 統計、衝突形状などを実行中に確認できます。

確認方法	対象	結果・用途
CPU 回帰テスト	セルキー、HP、CSV、武器間隔、乱数、SoundHandle	Debug / Release PASS
Scene Stress	Title → Play → Result を 3 周	各シーン 3 回訪問
SRV telemetry	使用数 / High-watermark	105 / 105
Debug x64 build	修正後の状態遷移実装を含む	2026-07-05 PASS

制作中に見直した点

対象	現在の構成
シーン生成	GameSceneFactory へ分離
ゲーム進行	GameplayFlowController へ分離
シーン間データ	GameSession へ集約
武器処理	PlayerWeaponController へ分離
衝突候補	空間分割で絞り、OBB で詳細判定
GPU リソース	FrameContext、Fence、状態追跡、ResourceBarrier
確認方法	CPU 回帰テストとシーン遷移ストレス

今後の改善

- PIX で Shadow Pass の GPU 時間、フレーム時間、描画数を同じ条件で記録する。
- 敵数・弾数を固定した比較で、空間分割が有効になる条件を CPU 時間として測定する。
- 起動後に追加したモデルをパス索引へ反映する更新処理と、非同期ロードを検討する。
- 管理クラスごとに異なる ID を取り違えないよう、型付きハンドルを広げる。
- CSV スキーマとゲーム進行を GPU なしで確認できる fixture を増やす。

現在の制約	Windows / DirectX 12 専用です。Shadow Map の状態遷移は実装済みですが、シルエットの最終確認と PIX による GPU 時間の定量評価は今後の検証項目です。
-------	---